

CSCE 465: Homework 2

Protect Agent Messages with Classic Cryptography

Zachary Renkema

October 2026

Contents

1	Lab Preparation (Task 0)	2
1.1	Versions	2
1.2	Screenshot of Output	2
2	Task 1	2
2.1	Design	2
2.2	Experiment: Replay	3
2.2.1	Output	3
2.3	What Happened	3
2.4	Root Cause	3
3	Task 2	3
3.1	Design	3
3.2	Evidence	4
3.3	Connection to Textbook Concepts	4
3.3.1	Diffie–Hellman Key Establishment	4
3.3.2	Digital Signatures	4
3.3.3	Entity Authentication	4
3.3.4	Nonces	4
3.3.5	Replay Resistance	4
3.3.6	Forward Secrecy	5
3.3.7	Key Separation	5
3.4	Man-in-the-Middle and Reflection Prevention	5
3.5	If the Signatures Are Removed	5
4	Adversarial Tests and Security Note (Task 4)	5
4.1	Test Summary	5
4.2	Test Run	5
4.3	Security Note	6
4.3.1	Attacker Model and Non-Goals	6
4.3.2	Why CTR Requires Unique IVs	7
4.3.3	Why Verify-Before-Decrypt Matters	7
4.3.4	Why Independent Encryption and MAC Keys Are Used	7
4.3.5	Comparison with an AEAD Mode (AES-GCM)	7
4.3.6	Why a Valid Protected Record May Still Carry a Dangerous Authorized Tool Call	7

5	AI Use	8
6	Appendix	8
6.1	File Index	8
6.2	Screenshots	9

1 Lab Preparation (Task 0)

1.1 Versions

% TODO: paste command and output

1.2 Screenshot of Output

```

csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/csce465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/csce465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$

```

Figure 1: Lab preparation: Python, OpenSSL, and package versions.

2 Task 1

2.1 Design

The design of this approach was taking the provided input command and encrypting it with AES with mode CTR. Note, as per the assignment description, there was no MAC. We call the cryptography library and utilize the AES encrypt and decrypt functions.

AES is a symmetric block cipher where the block and receiver share a secret key. It works by fixing blocks of 16 bytes by turning each plaintext matrix into a ciphered 128 matrix in which you need the key to decrypt. In our case, we used a 32-byte key.

To demonstrate why encryption alone is insufficient, a relay function was used which takes only ciphertext. With the assumption that the bad-actor in question already knows the message for-

matting, the actor knows that the cmd, "READ", sits at the byte offset of 11. The relay function then XOR's the ciphered bytes to modify the ciphered command.

2.2 Experiment: Replay

2.2.1 Output

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Encrypted using AES mode CTR: b'\x85\x04EW\xc1\xa7\x04@J\xe0\x8f\xce\xe8\x1e\x95&\x87T\x8c\xba\xe3
\xb1\xe5\xab\xac-0\xb3\xbb\xa9q\xd2N\xae\xe9:'
Decrypted using Cipher decryptor: b'{"action":"READ","path":"notes.txt"}'
Relay function used to mod original cmd: b'\x85\x04EW\xc1\xa7\x04@J\xe0\x8f\xd1\xec\x14\x94&\x87T
x8c\xba\xe3\xb1\xe5\xab\xac-0\xb3\xbb\xa9q\xd2N\xae\xe9:'
Decrypted new text: b'{"action":"MAKE","path":"notes.txt"}'
ct ^ tampered: 1f040a01
READ ^ MAKE : 1f040a01
pt ^ new pt : 1f040a01
receiver processes copy #1: b'{"action":"MAKE","path":"notes.txt"}'
receiver processes copy #2: b'{"action":"MAKE","path":"notes.txt"}'
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Encrypted using AES mode CTR: b'\x85\x04EW\xc1\xa7\x04@J\xe0\x8f\xce\xe8\x1e\x95&\x87T\x8c\xba\xe3\xb1\xe5\xab\xac-0\xb3\xbb\xa9q\xd2N\xae\xe9:'
Decrypted using Cipher decryptor: b'{"action":"READ","path":"notes.txt"}'
Relay function used to mod original cmd: b'\x85\x04EW\xc1\xa7\x04@J\xe0\x8f\xd1\xec\x14\x94&\x87T\x8c\xba\xe3\xb1\xe5\xab\xac-0\xb3\xbb\xa9q\xd2N\xae\xe9:'
Decrypted new text: b'{"action":"MAKE","path":"notes.txt"}'
ct ^ tampered: 1f040a01
READ ^ MAKE : 1f040a01
pt ^ new pt : 1f040a01
receiver processes copy #1: b'{"action":"MAKE","path":"notes.txt"}'
receiver processes copy #2: b'{"action":"MAKE","path":"notes.txt"}'
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

Figure 2: Task 1: output of baseline_ctr.py.

2.3 What Happened

As mentioned in the prior section, the relay function XOR'd the four ciphered bytes with READ ^ MAKE. The receiver function then decrypts using the library's decrypt function to output the new message. Even after providing to the receiver twice, the output remained changed.

2.4 Root Cause

The root cause of this is because nothing checks for modification in the stream of the original message. CTR is a stream cipher which means that the ciphered text is on the same stream as the output. It goes from plaintext through XOR and when decrypted, it is XOR'd again. As a result, a different XOR can output an identical change in the plaintext which is why a key is not needed for the bad-actor.

3 Task 2

3.1 Design

The design is centered around the Party class which is used on both sides to instantiate a gateway and node. The group and params for diffie-hellman is drawn from the ffdhe3072.pem file. Additionally, RSA keys are 3072-bit with exponent 65537 with signatures using RSA-PSS with SHA-256.

In addition to this file, we have class methods `begin_session`, `construct_canonical_transcript()`, and `decode_transcript()` which adhere to the task requirements.

`construct_canonical_transcript()` encodes eight fields and groups them as headers, identities, dh values, and nonces in a struct. `decode_transcript()` first checks for a truncated prefix and other errors before decoding.

3.2 Evidence

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 handshake.py
handshake complete, session_id: ac45e71960717e55
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ ls
AI_USAGE.md  baseline_ctr.py  ffdhe3072.pem  handshake.py  __pycache__  README.md  screenshots  secure_record.py  tests
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 handshake.py
handshake complete, session_id: ac45e71960717e55
```

Figure 3: Task 2: output of `handshake.py`.

3.3 Connection to Textbook Concepts

3.3.1 Diffie–Hellman Key Establishment

In DH, both sides, the client and server share the same secret key without transferring it with the information as to avoid man in the middle attacks. Both the client and server choose a private value which raises the other’s private value to its own. In the code, this is implemented over the shared public values in the `.pem` file and the exchange function produces the shared value.

3.3.2 Digital Signatures

As mentioned, only the private key holder can produce a signature so anyone with access to the public key can check it. The node and gateway sign the sha encrypted role which binds the party to every value in the handshake. The `sign_transcript` function does this and is then verified that it is a member of the party in `verify_peer`.

3.3.3 Entity Authentication

In the code each party is built with the peer’s RSA public key and the identity it expects, and rejects anything else.

3.3.4 Nonces

Nonces are random generated 16 byte values generated by the `os` package. They go into the transcript so every hash is different between both parties.

3.3.5 Replay Resistance

As to avoid reuse of old messages, the old signature hides the old nonce and dh values as to fail the verification if attempted for reuse.

3.3.6 Forward Secrecy

The private dh values are new to each sessions as mentioned. Additionally, RSA keys only sign and never encrypt. As a result, if a bad-actor were to steal or get a hold of any of this information, they would not be able to replicate the private keys afterwards.

3.3.7 Key Separation

A master key holds the four keys and session id which all derive from hmac. One key does not expose another if an issue were to occur.

3.4 Man-in-the-Middle and Reflection Prevention

A man-in-the-middle attack is defined as a bad-actor who sits in between and can either modify or collect information or session data in transfer to a client or server. This implementation of DH prevents this as all private values are signed and hashed. If an attacker were to interfere with their own value, the handshake would fail. The code checks for identity, dh value, and nonce's.

3.5 If the Signatures Are Removed

If signatures were removed, a bad-actor can basically perform a man-in-the-middle attack. The DH becomes unencrypted and they can record or save sessions in both directions.

4 Adversarial Tests and Security Note (Task 4)

4.1 Test Summary

#	Case	Test Function	Asserted Safe Failure
1	Valid handshake and bidirectional messages	test_valid_handshake_and_bidirectional_messages	Both sides derive equal keys and each message decrypts to the original
2	Modified ciphertext	test_modified_ciphertext_is_rejected	BadTag; receive counter stays at 0
3	Modified authenticated header	test_modified_header_is_rejected	BadTag; receive counter stays at 0
4	Replayed record	test_replayed_record_is_rejected	SequenceError on the second delivery
5	Record reflected into the opposite direction	test_reflected_record_is_rejected test_reflected_record_with_rewritten_direction_is_rejected	WrongDirection BadTag
6	Incorrect RSA public key, invalid RSA-PSS transcript signature, or reflected handshake message	test_incorrect_rsa_public_key test_invalid_transcript_signature test_reflected_handshake_message	BadSignature BadSignature ReflectedMessage

Table 1: Adversarial tests and the failure each one asserts.

4.2 Test Run

```
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/csce465/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/csce465/csce465-agentsec/hw2
collected 9 items

tests/test_1.py::test1 PASSED [ 11%]
tests/test_2.py::test_modified_ciphertext_is_rejected PASSED [ 22%]
tests/test_3.py::test_modified_header_is_rejected PASSED [ 33%]
tests/test_4.py::test_replayed_record_is_rejected PASSED [ 44%]
tests/test_5.py::test_reflected_record_is_rejected PASSED [ 55%]
tests/test_5.py::test_reflected_record_with_rewritten_direction_is_rejected PASSED [ 66%]
tests/test_6.py::test_incorrect_rsa_public_key PASSED [ 77%]
tests/test_6.py::test_invalid_transcript_signature PASSED [ 88%]
tests/test_6.py::test_reflected_handshake_message PASSED [100%]

===== 9 passed in 0.64s =====
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ source ~/csce465-agentsec/.venv/bin/activate
cd ~/csce465-agentsec/hw2
python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/csce465/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/csce465/csce465-agentsec/hw2
collected 9 items

tests/test_1.py::test1 PASSED [ 11%]
tests/test_2.py::test_modified_ciphertext_is_rejected PASSED [ 22%]
tests/test_3.py::test_modified_header_is_rejected PASSED [ 33%]
tests/test_4.py::test_replayed_record_is_rejected PASSED [ 44%]
tests/test_5.py::test_reflected_record_is_rejected PASSED [ 55%]
tests/test_5.py::test_reflected_record_with_rewritten_direction_is_rejected PASSED [ 66%]
tests/test_6.py::test_incorrect_rsa_public_key PASSED [ 77%]
tests/test_6.py::test_invalid_transcript_signature PASSED [ 88%]
tests/test_6.py::test_reflected_handshake_message PASSED [100%]

===== 9 passed in 0.54s =====
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

Figure 4: Task 4: pytest run, all tests passing.

4.3 Security Note

4.3.1 Attacker Model and Non-Goals

In a scenario with an attacker model, the attacker’s ultimate objective is to intercept the traffic between two nodes in order to modify, obtain, or read sensitive data. Diffie-Hellman and other means of encryption are intended to restrict or prevent this. The non goals are preventing the attacker

from dropping traffic or judging whether a command is safe to run like in the last assignment.

4.3.2 Why CTR Requires Unique IVs

As shown in task one, a CTR requires unique IVs as CTR ciphertext is plaintext XOR keystream. Reusing an IV under the same key reuses the upstream and allowing XOR's to cancel out the encryption easily. By signing the sequence you make them unique so an easily reversal as specified is not possible or is heavily hindered. This is why unique IVs are needed.

4.3.3 Why Verify-Before-Decrypt Matters

Verifying before decryption matters because if you don't, then you may decrypt the message to someone wrong. It essentially serves as a second layer of defense because if the verification fails, then there is no chance for an attacker to receive the information even if they somehow are able to capture the information. As shown in task 1, an unauthenticated CTR ciphertext can be altered in multiple ways. The way that the diffie hellman implementation in task 2 was secure, was because even if the encrypted text was modified, it would be rejected as it would fail the verification step.

4.3.4 Why Independent Encryption and MAC Keys Are Used

Independent Encryption and MAC Keys Are Used because each key serves a function. If one key fails or has a weakness, then that doesn't translate to another keeping the retention of security. Using the same key for multiple interactions serve a dangerous function of allowing an attacker to capture messages and replicate them for future interactions. As mentioned earlier in the report, the keys derive from one master key via a distance hmac label.

4.3.5 Comparison with an AEAD Mode (AES-GCM)

If AES-GCM was used instead, encryption and authentication would happen in a single call with one key, rather than running a cipher and a MAC as two separate steps. The header would go in as associated data, meaning it gets authenticated but stays in plaintext, which is the same job the tag is over the header is doing now. The main benefit is that there's less to mess up, since the client wouldn't have to manage a second key for the MAC or make sure they verify before they decrypt. It doesn't remove every responsibility, though. Nonces still can't repeat especially when considering reusing one in GCM is especially bad. You would still need sequence numbers to stop replayed messages.

4.3.6 Why a Valid Protected Record May Still Carry a Dangerous Authorized Tool Call

The record layer only tells the node that a message really came from the gateway, that it wasn't changed along the way, and that it arrived in the right order. It has no way of knowing whether the command inside is actually safe to run. If the gateway is compromised, set up wrong, or tricked into sending something destructive, that command still shows up fully authenticated and passes every check on the node. Whether an authenticated command should be allowed to run is a different problem, one that has to be handled by authorization, policy, and sandboxing. This is the main point of the assignment: a secure channel protects the message, but it can't say whether the message should be trusted.

5 AI Use

AI Use is dictated into the AI_USAGE.md file.

6 Appendix

6.1 File Index

All files are relative to the `hw2/` directory of the submitted repository.

File	Contents
<i>Submission</i>	
<code>report.pdf</code>	
<code>README.md</code>	
<code>AI_USAGE.md</code>	
<i>Task 0 - lab preparation</i>	
<code>ffdhe3072.pem</code>	
<i>Task 1 - baseline CTR</i>	
<code>baseline_ctr.py</code>	
<i>Task 2 - handshake</i>	
<code>handshake.py</code>	
<i>Task 3 - record layer</i>	
<code>secure_record.py</code>	
<i>Task 4 - tests</i>	
<i>Screenshots</i>	
<code>lab0.png</code>	Task 0 version output
<code>task1.png</code>	Task 1 output
<code>task2.png</code>	Task 2 output
<code>task3.png</code>	Task 3 output
<code>task4.png</code>	Task 4 test run

6.2 Screenshots

```
csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2
(.venv) csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/csce465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/csce465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) csce465@csce465-VMware-Virtual-Platform: ~/csce465-agentsec/hw2$
```

Figure 5: Lab preparation output.

```
csce465@csce465-Vmware-Virtual-Platform: ~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Encrypted using AES mode CTR: b'\x85\x04EW\xc1\xa7\x040J\x0e\x8f\xce\x08\x01\xe1\x95d\x87T\x8c\xba\xe3\xbb\x1e\x5\xab\xac-0\xbb\xbb\xa9q\xd2N\xae\xe9:'
Decrypted using Cipher decryptor: b'{"action": "READ", "path": "notes.txt"}'
Relay function used to mod original cmd: b'\x85\x04EW\xc1\xa7\x040J\xe0\x8f\x1d\xec\x14\x94d\x87T\x8c\xba\xe3\xbb\x1e\x5\xab\xac-0\xbb\xbb\xa9q\xd2N\xae\xe9:'
Decrypted new text: b'{"action": "MAKE", "path": "notes.txt"}'
ct ^ tampered: 1f040a01
READ ^ MAKE : 1f040a01
pt ^ new pt : 1f040a01
receiver processes copy #1: b'{"action": "MAKE", "path": "notes.txt"}'
receiver processes copy #2: b'{"action": "MAKE", "path": "notes.txt"}'
csce465@csce465-Vmware-Virtual-Platform: ~/csce465-agentsec/hw2$
```

Figure 6: Task 1 output.

```
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ ls
AI_USAGE.md  baseline_ctr.py  ffdhe3072.pem  handshake.py  __pycache__  README.md  screenshots  secure_record.py  tests
csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python3 handshake.py
handshake complete, session id: ac45e71960717e55
```

Figure 7: Task 2 output.

```
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ ls
AI_USAGE.md baseline_ctr.py ffdhe3072.pem handshake.py __pycache__ README.md screenshots secure_record.py tests
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ python secure_record.py
gateway to node: 0101000000000000000100000024b1ff80059508744d0000000000000087e54f0adf131933863a99784050eba8691fdad1934b740cea263b31953f6ee698199c20de753e50e22
f42992698f75297663c28202933087e56184dee754faeb1f8bc
node: (1, b '{"action": "READ", "path": "notes.txt"}')
gateway: (2, b '{"status": "ok"}')
bitflip error: BadTag
replay error: SequenceError
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$
```

Figure 8: Task 3 output.

```

csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$ source ~/csce465-agentsec/.venv/bin/activate
cd ~/csce465-agentsec/hw2
python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/csce465/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/csce465/csce465-agentsec/hw2
collected 9 items

tests/test_1.py::test1 PASSED [ 11%]
tests/test_2.py::test_modified_ciphertext_is_rejected PASSED [ 22%]
tests/test_3.py::test_modified_header_is_rejected PASSED [ 33%]
tests/test_4.py::test_replayed_record_is_rejected PASSED [ 44%]
tests/test_5.py::test_reflected_record_is_rejected PASSED [ 55%]
tests/test_5.py::test_reflected_record_with_rewritten_direction_is_rejected PASSED [ 66%]
tests/test_6.py::test_incorrect_rsa_public_key PASSED [ 77%]
tests/test_6.py::test_invalid_transcript_signature PASSED [ 88%]
tests/test_6.py::test_reflected_handshake_message PASSED [100%]

===== 9 passed in 0.54s =====
(.venv) csce465@csce465-VMware-Virtual-Platform:~/csce465-agentsec/hw2$

```

Figure 9: Task 4 test run.